

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL :3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Constraint Based Problem

Code of Conduct:

I Yuvann Nithish R (192411267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yuvann Nithish R

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design a Pushdown Automaton that accepts the language

$$L = \{ a^n b^n \mid n \geq 1 \}$$

Constraint: You are allowed to use only one stack symbol (besides initial symbol)

2. CFG to PDA Conversion with Limited Productions

Given a Context-Free Grammar:

$$S \rightarrow aSb \mid ab$$

Construct an equivalent PDA.

Constraint: PDA must be constructed using only 2 states

3. PDA to CFG Conversion with Restricted Variables

Convert a given PDA into an equivalent CFG.

Constraint: The resulting CFG must use at most 3 variables (non-terminals)

4. Turing Machine with Limited Tape Movement

Design a Turing Machine to recognize the language

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

Constraint: The tape head can move only to the right (no left movement allowed)

5. Consider a language $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Constraint: Design a Turing Machine for L

Below are the **college-level solutions** for all five questions, with the given constraints respected wherever they are mathematically possible.

1. PDA for $(L = \{a^n b^n \mid n \geq 1\})$

Constraint

Only **one stack symbol** besides the initial stack symbol is allowed.

Let the stack symbols be:

[
 $\Gamma = \{A, Z_0\}$
]

where:

-

(A) = counting symbol

-

(Z_0) = initial/bottom-of-stack symbol

-

Idea

-

For every a , push one A .

-

When b starts, pop one A for every b .

-

Accept when all A s are removed.

-

-

Since ($n \geq 1$), the empty string must **not** be accepted.

-

PDA

[
 $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
]

where:

[
 $Q = \{q_0, q_1, q_f\}$
]

[
 $\Sigma = \{a, b\}$
]

[
 $\Gamma = \{A, Z_0\}$
]

[
 $F = \{q_f\}$
]

Transitions

For the first a :

[
 $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$
]

For every additional a :

[
 $\delta(q_0, a, A) = \{(q_0, AA)\}$
]

When b begins:

[
 $\delta(q_0, b, A) = \{(q_1, \epsilon)\}$
]

For remaining b s:

[
 $\delta(q_1, b, A) = \{(q_1, \epsilon)\}$
]

When the stack returns to (Z_0) :

[
 $\delta(q_1, \epsilon, Z_0) = \{(q_f, Z_0)\}$
]

Example

For:

[
aaabbb
]

Stack operations:

Initially: Z_0

$a \rightarrow AZ_0$
 $a \rightarrow AAZ_0$
 $a \rightarrow AA AZ_0$

$b \rightarrow AA AZ_0$
 $b \rightarrow AZ_0$
 $b \rightarrow Z_0$

$\epsilon \rightarrow q_f$

Therefore:

[
 $\boxed{\text{aaabbb is accepted}}$
]

But:

[
aabbb
]

is rejected because one A remains.

Final PDA

```
[
\boxed{
\begin{aligned}
(q_0,a,Z_0)&\rightarrow(q_0,AZ_0)\backslash \\
(q_0,a,A)&\rightarrow(q_0,AA)\backslash \\
(q_0,b,A)&\rightarrow(q_1,\epsilon)\backslash \\
(q_1,b,A)&\rightarrow(q_1,\epsilon)\backslash \\
(q_1,\epsilon,Z_0)&\rightarrow(q_f,Z_0) \\
\end{aligned}
}
```

Only one working stack symbol, A , is used.

2. CFG to PDA Conversion

Given:

```
[
S \rightarrow aSb \mid ab
]
```

Constraint

The PDA must use **only 2 states**.

We can construct a PDA with:

```
[
Q={q_0,q_f}
]
```

where:

-
- (q_0) = processing state
-
-
- (q_f) = final state
-

PDA Definition

[
 $P=(Q,\Sigma,\Gamma,\delta,q_0,Z_0,F)$
]

where:

[
 $Q=\{q_0,q_f\}$
]

[
 $\Sigma=\{a,b\}$
]

[
 $\Gamma=\{S,a,b,Z_0\}$
]

[
 $F=\{q_f\}$
]

Initially, push (S) above (Z_0):

[
 $\delta(q_0,\epsilon,Z_0)=$
 $\{(q_0,SZ_0)\}$
]

Production ($S \rightarrow aSb$)

Replace (S) by (aSb):

[
 $\delta(q_0,\epsilon,S)=$
 $\{(q_0,aSb)\}$
]

Production ($S \rightarrow ab$)

Replace (S) by (ab):

[
 $\delta(q_0,\epsilon,S)=$
 $\{(q_0,ab)\}$
]

Terminal matching

For a:

$$\begin{bmatrix} \delta(q_0, a, a) = \{(q_0, \epsilon)\} \end{bmatrix}$$

For b:

$$\begin{bmatrix} \delta(q_0, b, b) = \{(q_0, \epsilon)\} \end{bmatrix}$$

Finally, when input is consumed and (Z_0) is on the stack:

$$\begin{bmatrix} \delta(q_0, \epsilon, Z_0) = \{(q_f, Z_0)\} \end{bmatrix}$$

Example

For (aabb):

$$\begin{bmatrix} S \rightarrow aSb \end{bmatrix}$$
$$\begin{bmatrix} \rightarrow aabb \end{bmatrix}$$

The PDA can perform:

S
↓
aSb
↓
aabb
↓
match a
↓
match a
↓
match b
↓
match b
↓
accept

The language generated is:

$$L = \{a^n b^n \mid n \geq 1\}$$

Important point

Only **two states** are used:

$$Q = \{q_0, q_f\}$$

3. PDA to CFG Conversion

Given information is incomplete

The question says:

“Convert a given PDA into an equivalent CFG.”

But the actual **PDA transition diagram/table has not been provided.**

Therefore, an exact CFG cannot be constructed without knowing:

PDA states

Input symbols

Stack symbols

Transitions

Initial state

Final state

General conversion

For a PDA with states (q_i, q_j) , introduce variables of the form:

$$[q_i A q_j]$$

which represent strings that take the PDA from state (q_i), with (A) on top of the stack, to state (q_j), after removing (A).

For example, if:

$$\begin{bmatrix} \delta(q_i, a, A) = (q_k, BC) \end{bmatrix}$$

then productions are constructed by introducing intermediate states:

$$\begin{bmatrix} [q_i A q_j] \\ \rightarrow \\ a[q_k B q_l][q_l C q_j] \end{bmatrix}$$

for appropriate states (q_l).

Constraint: at most 3 variables

The resulting grammar must have:

$$\boxed{\text{at most 3 non-terminals}}$$

However, **this cannot be guaranteed for an arbitrary PDA without knowing the PDA**. The standard PDA-to-CFG construction can require many variables.

So for Question 3, the actual PDA must be supplied before an exact 3-variable CFG can be produced.

Answer to write in an assessment

"The PDA required for conversion is not specified in the question. Hence an exact equivalent CFG cannot be determined. The standard conversion introduces variables representing state-to-state stack reductions. The resulting grammar can then be simplified, if possible, to satisfy the maximum-three-variable constraint."

4. TM for $(L = \{ww \mid w \in \{0, 1\}^*\})$

Constraint

Tape head can move **only to the right**.

This constraint creates a fundamental problem.

This TM is impossible under the given constraint.

A Turing machine whose head can **only move right** cannot go back and compare the first half of the input with the second half.

For example:

```
[  
w=101  
]
```

Input:

```
[  
101101  
]
```

The machine needs to compare:

first half	second half
101	101
↑	↑

But after reading the first half, a right-only machine cannot return to the beginning.

Why?

A normal TM recognizing:

```
[  
L={ww}  
]
```

needs to:

- 1.

Determine the midpoint.

- 2.

- 3.

Remember/mark symbols of the first half.

- 4.
- 5.

Compare them with corresponding symbols in the second half.

- 6.
- 7.

Move back and forth across the tape.

- 8.

A right-only head cannot perform this back-and-forth comparison.

In fact, a one-tape machine whose head is restricted to only rightward movement has essentially the power of a finite automaton for language recognition, and therefore can recognize only **regular languages**.

But:

[
 $\boxed{\{ww \mid w \in \{0,1\}^*\} \text{ is not regular}}$
]

Therefore:

[
 $\boxed{\text{No right-moving-only TM can recognize } L=\{ww\}.}$
]

Final answer

[
 $\boxed{\text{The design is impossible under the stated constraint.}}$
]

If left movement were allowed, a standard TM could recognize (ww) by marking and comparing corresponding symbols.

5. TM for $(L=\{a^n b^n c^n \mid n \geq 1\})$

This language is not context-free, but it **can be recognized by a Turing Machine**.

Idea

Use three marker symbols:

•

(X) for a matched a

•

•

(Y) for a matched b

•

•

(Z) for a matched c

•

For every a , the machine must find exactly one b and exactly one c .

Algorithm

Step 1

Find the leftmost unmarked a .

Replace:

[
 $a \rightarrow X$
]

Step 2

Move right until an unmarked b is found.

Replace:

[
 $b \rightarrow Y$
]

Step 3

Continue moving right until an unmarked c is found.

Replace:

$$[$$

$$c \rightarrow Z$$

$$]$$

Step 4

Move the head back to the beginning.

Step 5

Repeat the process.

Step 6

When no unmarked a remains, verify that only y s and z s are present.

If yes:

$$[$$

$$\boxed{\text{ACCEPT}}$$

$$]$$

Otherwise:

$$[$$

$$\boxed{\text{REJECT}}$$

$$]$$

Formal Definition

$$[$$

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

$$]$$

where:

$$[$$

$$Q =$$

$$\{q_0, q_1, q_2, q_3, q_4, q_{\text{accept}}, q_{\text{reject}}\}$$

$$]$$

Input alphabet:

$$[$$

$$\Sigma = \{a, b, c\}$$

$$]$$

Tape alphabet:

[
\\Gamma={a,b,c,X,Y,Z,B}
]

Initial state:

[
q_0
]

Blank symbol:

[
B
]

Final state:

[
F={q_{accept}}
]

Main transitions

Find an unmarked a

[
\\delta(q_0,a)=(q_1,X,R)
]

Search for b

[
\\delta(q_1,a)=(q_1,a,R)
]

[
\\delta(q_1,Y)=(q_1,Y,R)
]

[
\\delta(q_1,b)=(q_2,Y,R)
]

Search for c

[
 $\delta(q_2, b) = (q_2, b, R)$
]

[
 $\delta(q_2, Z) = (q_2, Z, R)$
]

[
 $\delta(q_2, c) = (q_3, Z, L)$
]

Move left

[
 $\delta(q_3, a) = (q_3, a, L)$
]

[
 $\delta(q_3, b) = (q_3, b, L)$
]

[
 $\delta(q_3, Y) = (q_3, Y, L)$
]

[
 $\delta(q_3, Z) = (q_3, Z, L)$
]

When x is reached:

[
 $\delta(q_3, X) = (q_0, X, R)$
]

Example: (aabbcc)

Initial tape:

a a b b c c

First cycle:

X a Y b Z c

Second cycle:

X X Y Y Z Z

Now there are no unmarked `as`.

The machine verifies the remaining symbols:

XXYYZZ

Therefore:

[
\boxed{aabbcc\text{ is accepted}}
]

Example: (aaabbbccc)

aaabbbccc
XaYbbccc
XXYYbZcc
XXXXXXXXZc
XXXXXXXXZZ

Thus:

[
\boxed{aaabbbccc\text{ is accepted}}
]

Final Summary

Q	Language	Constraint	Result
1	$(a^n b^n)$	Only one stack symbol	✓ Possible
2	$(S \rightarrow aSb \mid ab)$	Only 2 PDA states	✓ Possible
3	$PDA \rightarrow CFG$	≤ 3 variables	⚠ PDA not provided
4	(ww)	TM moves only right	✗ Impossible
5	$(a^n b^n c^n)$	Normal TM	✓ Possible

Important for your assessment: For **Q4**, don't try to invent a right-only TM. The correct theoretical answer is that the constraint makes the task **impossible**, because $(\{ww\})$ is non-regular while a one-way right-moving tape machine cannot recognize non-regular languages.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understandin g	30	Demonstrates clear and complete understand ing of the	Good understanding with minor gaps	Basic understanding with some misconception	Poor or incorrect understanding

		concept		s	
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand